

# Software Architecture Specification

Client Lead Management System

## Contents

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                    | <b>2</b>  |
| <b>2</b> | <b>Architectural Overview</b>                          | <b>2</b>  |
| 2.1      | Architectural Style . . . . .                          | 2         |
| 2.2      | Why This Style . . . . .                               | 4         |
| 2.3      | Server-Side Layering . . . . .                         | 4         |
| 2.4      | Client-Side Layering . . . . .                         | 4         |
| <b>3</b> | <b>Design Patterns and Techniques</b>                  | <b>5</b>  |
| 3.1      | Architectural Patterns . . . . .                       | 5         |
| 3.1.1    | Layered Architecture . . . . .                         | 5         |
| 3.1.2    | Service Layer . . . . .                                | 5         |
| 3.2      | Classical Design Patterns . . . . .                    | 5         |
| 3.2.1    | Chain of Responsibility — Middleware . . . . .         | 5         |
| 3.2.2    | Observer / Publish–Subscribe — Domain Events . . . . . | 5         |
| 3.3      | Application and UI Techniques . . . . .                | 5         |
| 3.3.1    | Optimistic Updates . . . . .                           | 5         |
| 3.3.2    | Server-State Caching . . . . .                         | 6         |
| 3.3.3    | Push-to-Cache Updates . . . . .                        | 6         |
| 3.3.4    | Controlled Components . . . . .                        | 6         |
| 3.3.5    | Shared Application Shell . . . . .                     | 6         |
| 3.4      | Supporting Techniques . . . . .                        | 6         |
| 3.4.1    | Boundary Validation . . . . .                          | 6         |
| 3.4.2    | Cascade Deletion . . . . .                             | 6         |
| 3.4.3    | Non-Blocking Secondary Operations . . . . .            | 6         |
| <b>4</b> | <b>Technology Stack</b>                                | <b>7</b>  |
| 4.1      | Summary . . . . .                                      | 7         |
| 4.2      | Justification . . . . .                                | 7         |
| <b>5</b> | <b>Deployment</b>                                      | <b>8</b>  |
| 5.1      | Topology . . . . .                                     | 8         |
| 5.2      | Deploy Targets . . . . .                               | 9         |
| 5.3      | Failure Domains and Accepted Risk . . . . .            | 9         |
| <b>6</b> | <b>Summary</b>                                         | <b>10</b> |

# 1 Introduction

This document describes how the Client Lead Management System is structured. It covers the architectural style, the design patterns and application techniques used, the technology stack, and the deployment topology.

It complements the Software Requirements Specification, which defines *what* the system must do. This document defines *how* it is built. Endpoint-level detail lives in the API Contract.

The architecture view described in Section 2 and Section 3 answers *what the parts are and how they depend on each other*; the deployment view in Section 5 answers *where the parts run and what fails together*.

## 2 Architectural Overview

### 2.1 Architectural Style

The system follows a **three-package layered architecture**, with a clear separation between presentation, application logic, and data.

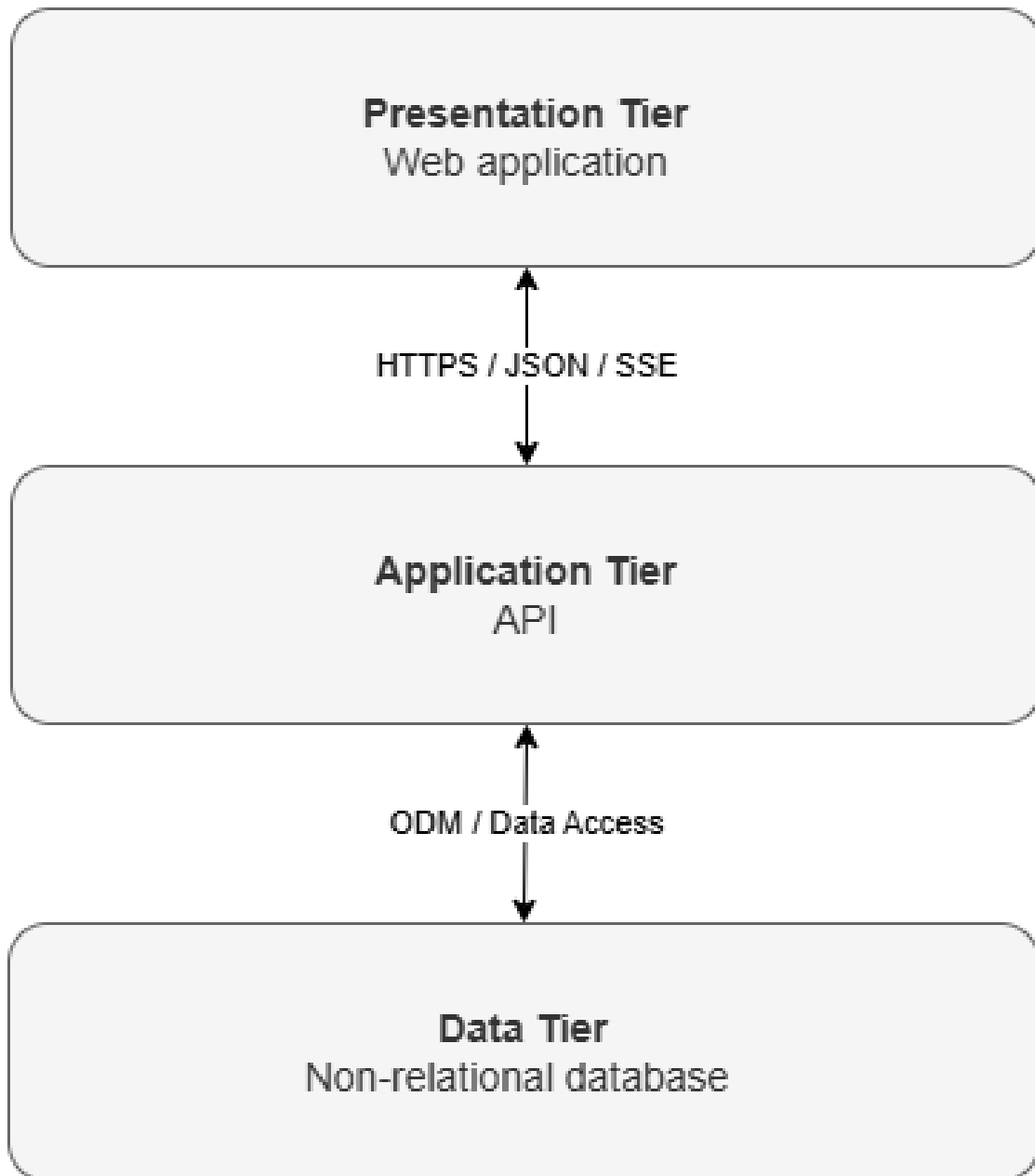


Figure 1: Logical architecture. Packages group related components; dependencies between packages are shown as **use** relationships.

The **presentation** package contains the web application, the API client, and the event stream subscriber. The **application** package contains the HTTP router, authentication middleware, controllers, services, and the event stream broadcaster. The **data** package contains the data access layer and the external document store.

Each package is independently deployable and communicates only over defined interfaces. The presentation package never accesses the document store directly; all reads and writes pass through the application package.

The package boundaries correspond to the deployment boundaries documented in Section 5: the presentation package runs in the browser, the application package runs as the API runtime, and the data package is a managed external service.

## 2.2 Why This Style

Three reasons drove the choice:

1. **Fit for the problem.** The system is a straightforward CRUD application with authentication and a single realtime channel. A layered style is a natural match—it maps directly to the responsibility boundaries (render, decide, persist) and avoids the ceremony of a microservice split that would serve no purpose at this scale.
2. **Familiar and reviewable.** Layered architectures are widely used for web applications. A reviewer can understand the flow from the folder structure and architecture diagram without needing to understand a distributed microservice system.
3. **Testable in isolation.** Because business logic lives in its own layer, it can be unit-tested without spinning up a full server, and the data layer can be changed without touching the presentation layer.

## 2.3 Server-Side Layering

The application package is itself layered into five responsibilities:

| Layer              | Responsibility                                                                      | Depends on  |
|--------------------|-------------------------------------------------------------------------------------|-------------|
| <b>Routes</b>      | Bind HTTP method and path to a controller. No business logic.                       | Controllers |
| <b>Controllers</b> | Parse the request, call a service, and shape the response.                          | Services    |
| <b>Services</b>    | Domain logic: creating leads, transitioning status, and handling domain operations. | Models      |
| <b>Models</b>      | Schema definition, validation at the persistence boundary, and indexes.             | Data access |
| <b>Middleware</b>  | Cross-cutting concerns such as authentication, validation, and error handling.      | —           |

Table 1: Server-side responsibilities within the application package

This separation keeps business logic in the service layer, allowing it to be tested without requiring a full HTTP request or a live database.

## 2.4 Client-Side Layering

The presentation package is organised into four responsibilities:

| Layer             | Responsibility                                                                                                  |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Pages</b>      | Route-level composition. Own data fetching and layout.                                                          |
| <b>Components</b> | Presentational, reusable, and without direct data access.                                                       |
| <b>Hooks</b>      | Shared logic such as authentication state, realtime subscription, and form handling.                            |
| <b>API client</b> | A single module that owns requests to the backend, so credentials and the base URL are configured consistently. |

Table 2: Client-side responsibilities within the presentation package

## 3 Design Patterns and Techniques

The system uses a combination of architectural patterns, established design patterns, and application-level techniques. These are separated below to distinguish the structural patterns from implementation and UI techniques.

### 3.1 Architectural Patterns

#### 3.1.1 Layered Architecture

The system follows a three-package architecture consisting of the presentation, application, and data packages. Within the application package, routes delegate to controllers, controllers delegate to services, and services delegate to models. This separates presentation, business logic, and persistence concerns.

#### 3.1.2 Service Layer

Business operations are encapsulated within service functions rather than being implemented directly inside controllers. This keeps controllers thin and allows domain logic to be tested independently of HTTP handling.

### 3.2 Classical Design Patterns

#### 3.2.1 Chain of Responsibility — Middleware

Express middleware forms a processing chain through which requests pass before reaching their final handler. Authentication, validation, and error handling can therefore be added as independent processing steps without embedding these concerns into every controller.

#### 3.2.2 Observer / Publish–Subscribe — Domain Events

The application uses an event-driven mechanism in which services emit domain events such as `lead.created` and `activity.created`, while the event stream broadcaster subscribes to these events. This separates the component producing an event from the components interested in reacting to it.

### 3.3 Application and UI Techniques

#### 3.3.1 Optimistic Updates

The client updates its local query state immediately when a mutation occurs, while the request is processed by the server. If the request fails, the previous state is restored. This improves perceived responsiveness.

### **3.3.2 Server-State Caching**

Remote data is managed as server state using a query and cache layer. Components consume cached data rather than independently managing loading, error, and data states.

### **3.3.3 Push-to-Cache Updates**

Realtime events are applied directly to the client-side cache instead of triggering a full refetch. This reduces unnecessary requests while keeping the interface up to date.

### **3.3.4 Controlled Components**

Form inputs are controlled by React state, providing a single source of truth for their values and validation state.

### **3.3.5 Shared Application Shell**

Authenticated pages share a common shell containing navigation and global UI elements. Individual screens therefore inherit common functionality without duplicating it.

## **3.4 Supporting Techniques**

### **3.4.1 Boundary Validation**

Incoming requests are validated before reaching business logic. This ensures malformed input is rejected consistently before persistence or domain operations occur.

### **3.4.2 Cascade Deletion**

Deleting a lead also removes its associated notes and activity records through model-level hooks. The cascade is kept close to the persistence model so related records are not left behind when a lead is removed.

### **3.4.3 Non-Blocking Secondary Operations**

Failures in secondary functionality are handled without failing the primary operation. Activity logging, for example, does not prevent a successful lead mutation from completing. The primary operation is treated as authoritative, and secondary operations are allowed to fail without escalating.

## 4 Technology Stack

### 4.1 Summary

| Layer / Concern           | Technology                                 |
|---------------------------|--------------------------------------------|
| <b>Frontend framework</b> | React 18+                                  |
| <b>Client language</b>    | TypeScript                                 |
| <b>Build tool</b>         | Vite                                       |
| <b>Styling</b>            | Tailwind CSS                               |
| <b>Routing</b>            | React Router                               |
| <b>Server state</b>       | TanStack Query                             |
| <b>Backend runtime</b>    | Node.js 20 LTS                             |
| <b>Backend framework</b>  | Express                                    |
| <b>Database</b>           | MongoDB Atlas M0                           |
| <b>Data access</b>        | Mongoose (ODM)                             |
| <b>Validation</b>         | Zod                                        |
| <b>Authentication</b>     | JWT in httpOnly cookie, bcrypt for hashing |
| <b>Realtime</b>           | Server-Sent Events                         |
| <b>Frontend hosting</b>   | Vercel Hobby                               |
| <b>Backend hosting</b>    | Render Free                                |
| <b>Version control</b>    | Git / GitHub                               |

Table 3: Technology stack

### 4.2 Justification

**React + Vite.** React was chosen over a vanilla or server-rendered approach because the dashboard has meaningful client state, including filters, optimistic updates, and realtime merges, that would be awkward to manage otherwise. Vite provides a fast development and build workflow.

**Tailwind CSS.** Utility-first styling keeps styles co-located with markup and supports the application's responsive and dark-mode requirements without requiring a large collection of separate stylesheets.

**TanStack Query.** TanStack Query provides fetching, caching, invalidation, refetching, and optimistic-update support. It gives the client a consistent mechanism for normal mutations and realtime-driven cache updates.

**Node.js + Express.** Node allows the backend to share a language with the frontend, reducing context switching. Express provides a small and readable API layer without requiring the additional structure of a heavier framework.

**MongoDB + Mongoose.** The data model is document-shaped, centred around leads with associated notes and activities. MongoDB is therefore a natural fit, while Mongoose provides schema definitions, validation, and model hooks used by the application.

**Zod.** Zod provides declarative, TypeScript-friendly schema validation and keeps request validation close to the API boundary.

**JWT in httpOnly cookie.** The session credential is stored in an httpOnly cookie so that it is not readable by client-side JavaScript. Cookie and CORS configuration are used to support authenticated cross-origin requests between the deployed frontend and backend.

**Server-Sent Events.** Realtime traffic is one-way, from server to client. SSE is a native browser API and fits the application’s broadcast-oriented realtime requirements without requiring bidirectional WebSocket communication.

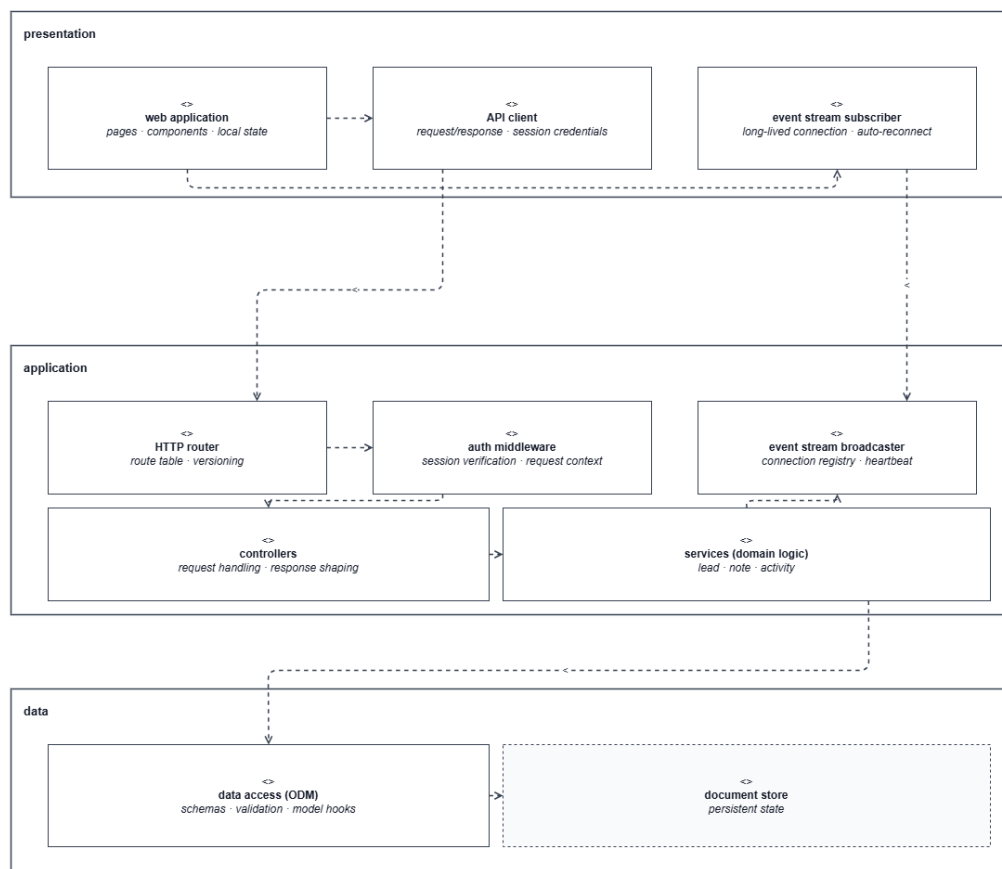
**Vercel + Render + Atlas.** The frontend, backend, and database are deployed using managed services that provide suitable free tiers for the project. This avoids unnecessary server administration while maintaining a deployment model close to local development.

## 5 Deployment

### 5.1 Topology

The system consists of three independently deployed services, with deployments triggered from the project’s Git repository.

System: FUTURE\_FS\_02 | View: logical architecture



Owner: Zelamene Shazi · Source: docs/architecture.drawio · Last verified: 2026-09-11

Figure 2: Deployment topology for the production environment.

The frontend and backend are deployed as public services. The database is reachable only by the backend. Two trust boundaries are shown on the figure: the boundary between the public

internet and the private workload network, and the boundary between the application package and the data package, which are in separate credential domains.

The frontend and backend are served from different origins, which means cross-origin requests and cookie attributes must be configured explicitly for session authentication to work. This is an architectural consequence of the split deployment, not a preference, and it is one of the failure modes most likely to be rediscovered if the topology is ever changed.

The backend runs on a hosting tier that suspends the service after a period of inactivity, and the realtime channel is a long-lived connection that the hosting platform’s proxy will close if it is idle. Both constraints shape the architecture in specific ways: a scheduled external health check keeps the service warm, the client displays a waking-up indicator rather than appearing to hang, and the server emits a heartbeat on the realtime channel to keep the connection alive. Configuration details for these measures are in the deployment and operations notes.

## 5.2 Deploy Targets

| Service  | Platform         | Role                                                                                                                 |
|----------|------------------|----------------------------------------------------------------------------------------------------------------------|
| Frontend | Vercel (Hobby)   | Serves the compiled single-page application as static assets. Stateless and edge-cached; hosts no application logic. |
| Backend  | Render (Free)    | Runs the API as a single instance, from which all reads and writes originate.                                        |
| Database | MongoDB Atlas M0 | Managed document store; the sole persistent state in the system. Reachable only by the backend.                      |

Table 4: Deployment targets

## 5.3 Failure Domains and Accepted Risk

The deployment has a single failure domain at each of the two stateful layers, and this is documented rather than hidden.

- **API runtime.** A single instance, with no horizontal scale. If the instance is lost, the API is unavailable until the platform restores it.
- **Database.** A single-node cluster, with no standby in a second failure domain. Daily backup provides an effective recovery point objective of approximately twenty-four hours.
- **Region affinity.** Whether the API runtime and the database share an underlying availability zone has not been verified. If they do, a single-zone failure would take both down together.

These characteristics are accepted for a project of this scope, where the objective is a working system at zero cost rather than a production-grade availability target. They are recorded here because a failure-domain characteristic that is not documented cannot be reviewed, and a review that cannot see the constraint cannot decide whether to accept it.

## 6 Summary

| Aspect                        |              | Choice                                                                                                           |
|-------------------------------|--------------|------------------------------------------------------------------------------------------------------------------|
| <b>Architectural style</b>    |              | Three-package layered architecture                                                                               |
| <b>Server layering</b>        |              | Routes → Controllers → Services → Models                                                                         |
| <b>Client layering</b>        |              | Pages, Components, Hooks, API client                                                                             |
| <b>Classical patterns</b>     |              | Chain of Responsibility, Observer / Publish–Subscribe                                                            |
| <b>Application techniques</b> | <b>tech-</b> | Optimistic updates, server-state caching, push-to-cache updates, controlled components, shared application shell |
| <b>Persistence techniques</b> | <b>tech-</b> | Boundary validation, cascade deletion, non-blocking secondary operations                                         |
| <b>Realtime</b>               |              | Server-Sent Events                                                                                               |
| <b>Authentication</b>         |              | JWT in httpOnly cookie                                                                                           |
| <b>Deployment</b>             |              | Frontend, backend, and database as managed services                                                              |
| <b>Failure model</b>          |              | Single failure domain per stateful layer; documented and accepted (Section <a href="#">5.3</a> )                 |

Table 5: Architecture summary